

CYBER SECURITY MINOR PROJECT REPORT

Pentesting on ColdBox

Submitted By

Name: Prince Perinbaraj S

Department : CSE

College: Rajalakshmi Engineering College

Academic Year: 2022 - 2026

Table of Contents

1. Abstract
2. Introduction
3. Objectives
4. Scope of the Project
5. Tools Used
6. Testing Environment
7. Methodology
8. Reconnaissance Phase
9. Vulnerability Scanning
10. Manual Penetration Testing
11. Vulnerabilities Identified
12. Risk Analysis
13. Recommendations
14. Result and Discussion
15. Future Enhancements
16. Conclusion
17. References

1. Abstract

The “Pentesting on ColdBox” project was conducted to assess the security posture of a ColdBox web application and identify vulnerabilities that could potentially be exploited by attackers. The project involved both automated and manual penetration testing techniques to evaluate the application’s security controls and identify weaknesses.

The testing methodology included reconnaissance, vulnerability scanning, exploitation testing, session analysis, and risk assessment. Automated tools such as Nmap, OWASP ZAP, Burp Suite, SQLMap, and Nikto were used during the testing process. Manual testing techniques were also applied to verify vulnerabilities and analyze application behavior.

During the assessment, multiple vulnerabilities were identified, including SQL Injection, Cross Site Scripting (XSS), missing security headers, and session management weaknesses. These vulnerabilities were analyzed to determine their severity, impact, and possible remediation methods.

The project demonstrates the importance of regular security testing in web applications and highlights how improper input validation and weak security configurations can expose applications to cyber attacks. Recommendations were provided to improve the security posture of the ColdBox application and reduce the overall risk of exploitation.

2. Introduction

Web applications have become an essential component of modern organizations. Businesses rely heavily on web technologies to provide services, manage operations, and communicate with users. However, the increasing use of web applications has also increased the number of cyber attacks targeting these systems.

Penetration testing is a cybersecurity practice used to identify vulnerabilities in a system before malicious attackers can exploit them. It simulates real-world attacks in a controlled environment to evaluate the effectiveness of existing security measures.

ColdBox is an open-source web application framework based on CFML (ColdFusion Markup Language). It is widely used for developing modular and scalable web applications. Although ColdBox includes several built-in security mechanisms, applications developed using the framework may still contain vulnerabilities due to insecure coding practices or improper configuration.

The purpose of this project is to perform penetration testing on a ColdBox-based application to identify potential security weaknesses and recommend suitable mitigation techniques.

3. Objectives

The main objectives of this project are:

- To understand the fundamentals of penetration testing.
- To analyze the security posture of a ColdBox web application.
- To identify vulnerabilities such as SQL Injection and Cross Site Scripting.
- To evaluate the effectiveness of existing security controls.
- To recommend mitigation strategies for identified vulnerabilities.
- To gain practical knowledge of penetration testing tools and methodologies.

4. Scope of the Project

This project focuses on penetration testing of a locally hosted ColdBox web application in a controlled laboratory environment. The testing activities were limited to authorized systems and were performed only for educational and research purposes.

The scope of the project includes:

- Information gathering and reconnaissance
- Vulnerability scanning
- Manual penetration testing
- Session management analysis
- Security assessment and reporting

The project does not include:

- Attacks against real-world production systems
- Denial of Service (DoS) attacks
- Unauthorized exploitation of external systems

5. Tools Used

Tool Name	Purpose
Kali Linux	Penetration testing operating system
Nmap	Network and port scanning
OWASP ZAP	Web application vulnerability scanning
Burp Suite	Intercepting and modifying HTTP requests
SQLMap	SQL Injection testing
Nikto	Web server vulnerability scanning
Wappalyzer	Technology stack identification

6. Testing Environment

The testing environment was configured in a local laboratory setup using Kali Linux. The ColdBox application was hosted locally to ensure safe and authorized testing.

Hardware Requirements

- Processor: Intel/AMD Processor
- RAM: Minimum 8GB
- Storage: Minimum 20GB Free Space

Software Requirements

- Kali Linux
- Java JDK
- CommandBox
- ColdBox Framework
- Penetration Testing Tools

Network Environment

The application was tested on localhost using the IP address:

127.0.0.1

This ensured that all penetration testing activities remained within a secure and isolated environment.

7. Methodology

The penetration testing process followed a structured methodology consisting of the following phases:

7.1 Reconnaissance

Information gathering was performed to identify the target application structure, technologies, open ports, and possible entry points.

7.2 Scanning

Automated tools were used to scan the application and identify known vulnerabilities and misconfigurations.

7.3 Exploitation

Manual testing techniques were used to exploit identified vulnerabilities such as SQL Injection and XSS.

7.4 Post Exploitation

The impact of successful exploitation was analyzed to understand the level of risk associated with each vulnerability.

7.5 Reporting

All findings were documented along with remediation recommendations.

8. Reconnaissance Phase

Reconnaissance is the process of gathering information about the target application before attempting exploitation.

8.1 Network Scanning Using Nmap

Nmap was used to identify open ports and running services on the target system.

Command Used

```
nmap -sV 127.0.0.1
```

Purpose

- Identify open ports
- Detect running services
- Determine service versions

Findings

The scan identified active HTTP services running on the target system.

8.2 Technology Identification

Wappalyzer was used to identify the technologies used in the application.

Technologies Identified

- ColdBox Framework
- CFML
- Apache Web Server
- Java Runtime Environment

8.3 Entry Point Identification

The following application components were identified as potential attack surfaces:

- Login Forms
- Search Fields
- URL Parameters
- Session Cookies
- User Input Forms

9. Vulnerability Scanning

Automated vulnerability scanning was performed using Nikto and OWASP ZAP.

9.1 Nikto Scan

Nikto was used to identify web server vulnerabilities and misconfigurations.

Command Used

```
nikto -h http://127.0.0.1
```

Findings

- Missing security headers
- Server information disclosure
- Potential configuration weaknesses

9.2 OWASP ZAP Scan

OWASP ZAP was used to perform spider scanning and active vulnerability testing.

Scan Types Performed

- Spider Scan
- Passive Scan
- Active Scan

Vulnerabilities Identified

- Missing X-Frame-Options Header
- Cookie Security Weaknesses
- Potential XSS Vulnerabilities
- Information Disclosure

9.3 Burp Suite Analysis

Burp Suite was used to intercept and analyze HTTP requests and responses.

Activities Performed

- Request interception
- Parameter manipulation
- Session analysis
- Cookie inspection

10. Manual Penetration Testing

Manual penetration testing was conducted to verify vulnerabilities identified during automated scanning.

10.1 SQL Injection Testing

SQL Injection testing was performed on login forms and URL parameters.

Payload Used

' OR 1=1 --

Objective

The objective was to determine whether user input was improperly sanitized before being executed as SQL queries.

Result

The application showed signs of improper input validation, indicating possible SQL Injection vulnerabilities.

Impact

Successful exploitation could allow attackers to:

- Access sensitive database information
- Modify records
- Bypass authentication
- Execute unauthorized queries

10.2 Cross Site Scripting (XSS)

Cross Site Scripting testing was performed by injecting JavaScript payloads into user input fields.

Payload Used

```
<script>alert('XSS')</script>
```

Result

The application reflected unsanitized input back to the browser, indicating possible XSS vulnerabilities.

Impact

Attackers could potentially:

- Steal session cookies
- Redirect users to malicious websites
- Execute malicious scripts in the victim's browser

10.3 Session Management Testing

Session cookies were analyzed using Burp Suite.

Findings

- Missing HttpOnly Flag
- Missing Secure Attribute
- Weak Session Handling

Impact

Weak session management could allow attackers to hijack user sessions and impersonate authenticated users.

10.4 Authentication Testing

Authentication testing was performed to evaluate password policies and login error handling.

Findings

- Weak password combinations accepted
- Informative error messages observed

Impact

Weak authentication controls increase the risk of unauthorized access.

11. Vulnerabilities Identified

Vulnerability	Risk Level	Description
SQL Injection	High	Unsanitized input allowed SQL query manipulation
Cross Site Scripting (XSS)	High	User input reflected without proper encoding
Session Fixation	Medium	Weak session handling mechanisms identified
Missing Security Headers	Medium	Important HTTP security headers not configured
Information Disclosure	Low	Server details exposed in responses

12. Risk Analysis

The identified vulnerabilities were analyzed based on their severity and potential impact.

High Risk Vulnerabilities

SQL Injection

This vulnerability can lead to complete database compromise and unauthorized access to sensitive information.

Cross Site Scripting (XSS)

XSS vulnerabilities can be exploited to steal user sessions and execute malicious scripts.

Medium Risk Vulnerabilities

Session Fixation

Attackers may hijack active sessions and impersonate legitimate users.

Missing Security Headers

The absence of secure headers increases exposure to attacks such as clickjacking.

Low Risk Vulnerabilities

Information Disclosure

Server information leakage may help attackers identify technologies and plan targeted attacks.

13. Recommendations

13.1 SQL Injection Prevention

- Use parameterized queries
- Implement prepared statements
- Validate all user input
- Avoid dynamic SQL queries

13.2 XSS Prevention

- Encode output properly
- Sanitize user input
- Implement Content Security Policy (CSP)
- Avoid directly rendering user input

13.3 Session Security Improvements

- Enable HttpOnly cookies
- Enable Secure cookie attributes
- Regenerate session IDs after login
- Use strong session management policies

13.4 Server Hardening

- Hide server version information
- Configure secure HTTP headers
- Regularly update frameworks and dependencies
- Disable unnecessary services

13.5 Authentication Improvements

- Enforce strong password policies
- Implement account lockout mechanisms
- Use multi-factor authentication where possible

14. Result and Discussion

The penetration testing process successfully identified several vulnerabilities in the ColdBox application environment. Both automated and manual testing methods proved effective in discovering security weaknesses.

The project demonstrated how improper input validation, weak session handling, and missing security controls can expose web applications to cyber threats. The findings emphasize the importance of secure coding practices, regular vulnerability assessments, and continuous monitoring.

The recommendations provided in this report can significantly improve the application's security posture and reduce the likelihood of successful cyber attacks.

15. Future Enhancements

Future improvements for this project may include:

- Integration of automated security testing into CI/CD pipelines
- Cloud-based penetration testing environments
- Advanced API security testing
- Security monitoring and logging integration
- Testing against OWASP Top 10 vulnerabilities

- Implementation of automated remediation systems

16. Conclusion

The “Pentesting on ColdBox” project provided valuable insights into the security posture of a ColdBox web application. The project successfully identified multiple vulnerabilities using both automated and manual penetration testing techniques.

The identified vulnerabilities included SQL Injection, Cross Site Scripting, session management weaknesses, and missing security configurations. Recommendations were provided to mitigate these vulnerabilities and improve the application’s overall security.

This project highlights the importance of regular security assessments and demonstrates how penetration testing can help organizations proactively identify and address security weaknesses before they can be exploited by attackers.

Overall, the project enhanced practical understanding of web application security, penetration testing methodologies, and cybersecurity best practices.

17. References

1. OWASP Official Documentation
2. Nmap Documentation
3. Burp Suite Documentation
4. OWASP ZAP User Guide
5. SQLMap Official Documentation
6. ColdBox Framework Documentation
7. Nikto Documentation
8. Kali Linux Tools Documentation

Appendix

Appendix A – Example Commands

Nmap Scan

```
nmap -sV 127.0.0.1
```

Nikto Scan

nikto -h <http://127.0.0.1>

SQLMap Scan

sqlmap -u "<http://127.0.0.1/product?id=1>" --dbs

Gobuster Directory Scan

gobuster dir -u <http://127.0.0.1> -w /usr/share/wordlists/dirb/common.txt

Appendix B – Screenshots

(Add the following screenshots here)

1. ColdBox Application Running
2. Nmap Scan Result
3. Nikto Scan Result
4. OWASP ZAP Alerts
5. Burp Suite Interception
6. SQLMap Result
7. XSS Alert Popup
8. Session Cookie Analysis